STATE LEVEL HACKATHON HOSPITAL EMERGENCY RESPONSE COORDINATOR

Ranking emergency patients in real time on QNX RTOS

NARAYANA ENGINEERING COLLEGE, NELLORE, ANDHRA PRADESH

SUBMITTED BY:

ATHIPATI MADHU SRI

(23711A0404)

BUTTI NAVYA SAI REDDY

(23711A0407)

SHAIK SHABREEN

(24715A0406)

ABSTRACT

The **Hospital Emergency Response Coordinator using QNX RTOS** prioritizes emergency patients based on their severity. It uses **five independent processes** for triage, priority ranking, ICU bed management, and dashboard monitoring. Patients are classified into **four severity levels**, ensuring critical patients receive higher priority. The system aims to improve **response time, patient management, and hospital resource utilization** through real-time processing and inter-process communication.

Introduction

The **Hospital Emergency Response Coordinator** using QNX RTOS is designed to improve emergency patient management by prioritizing patients according to the severity of their medical conditions. During emergencies, multiple patients may require immediate attention, making it important to identify those who need treatment first. The system uses separate processes for patient triage, priority ranking, ICU bed management, and dashboard display. It provides an organized view of patient priorities and available resources. By using the QNX Real-Time Operating System (RTOS), the project aims to support timely updates, efficient resource utilization, and better coordination in emergency situations.

Problem Statement

Managing multiple emergency patients at the same time can delay treatment and make resource allocation difficult. A system is needed to prioritize patients quickly and monitor available hospital resources.

Objectives

The main objective of this project is to develop a real-time hospital management prototype that supports emergency patient prioritization and resource monitoring.

- To classify patients into four severity levels: Critical, Severe, Moderate, and Mild.
- To arrange patients in a priority queue, placing the most severe cases first.
- To monitor the availability of ICU beds and hospital resources.
- To update patient rankings when their severity or resource availability changes.
- To display patient priorities and the next patient to be treated on a dashboard.
- To implement separate processes using QNX RTOS and Inter-Process Communication (IPC).
- To support faster decision-making and improve the organization of emergency patient management.

Existing System and proposed System

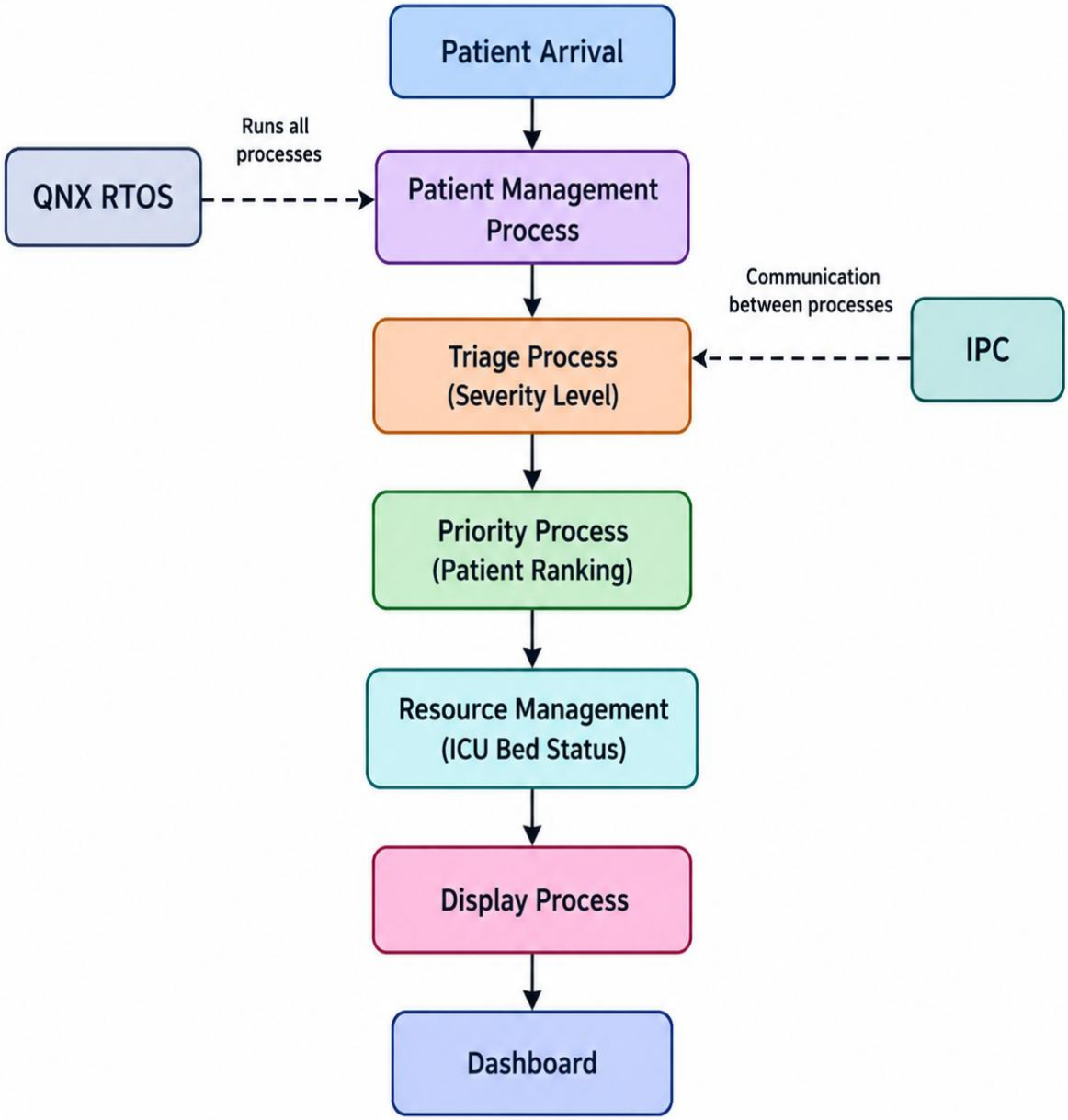
The existing hospital management system may rely on manual patient assessment, priority updates, and ICU bed tracking, making it challenging to manage multiple emergency patients simultaneously. To address these challenges, the proposed **Hospital Emergency Response Coordinator** using QNX RTOS uses five independent processes for patient management, triage, priority ranking, resource management, and dashboard display. It classifies patients into Critical, Severe, Moderate, and Mild levels and ranks them according to severity. The system also monitors ICU bed availability and displays updated patient priorities through a dashboard. By using QNX RTOS and Inter-Process Communication (IPC), the proposed system aims to improve patient prioritization, resource monitoring, and timely decision-making during emergencies**.**

S.No.	Existing System	Proposed System
1	Manual patient prioritization	Severity-based priority ranking
2	Manual status updates	Timely patient status updates
3	Separate ICU bed tracking	Dedicated resource management
4	Separate patient records	Centralized dashboard monitoring
5	Conventional workflow	Five independent processes using QNX RTOS and IPC

System architecture and block diagram

The **Hospital Emergency Response Coordinator** uses QNX RTOS and five independent processes to manage emergency patients and hospital resources. Each process performs a specific task and communicates with other processes using Inter-Process Communication (IPC).

- Patient Management Process: Handles patient arrival and patient information.
- Triage Process: Assigns severity levels based on patient condition.
- Priority Process: Ranks patients from highest to lowest severity.
- Resource Management Process: Tracks ICU bed availability.
- Display Process: Shows patient priorities and resource status on the dashboard


Patient Arrival
Runs all processes
QNX RTOS
Patient Management Process
Communication between processes
IPC
Triage Process (Severity Level)
Priority Process (Patient Ranking)
Resource Management (ICU Bed Status)
Display Process
Dashboard

Software and Hardware Requirements

S.No.	Software	Purpose
1	QNX Neutrino RTOS	Real-time operating system
2	C / C++ Compiler	Developing and compiling the program
3	QNX Software Development Platform (QNX SDP)	Application development and debugging
4	QNX Momentics IDE	Writing, building, and debugging code
5	Inter-Process Communication (IPC)	Communication between independent processes
6	Dashboard / Display Interface	Displaying patient priorities and ICU bed status

. **Hardware Requirements**

S.No.	Hardware	Purpose
1	Computer / Laptop	Developing and running the system
2	RAM (Minimum 4 GB recommended)	Supporting the development environment
3	Processor (Intel / AMD)	Executing system processes
4	Storage (Minimum 10 GB free recommended)	Storing project files and development tools
5	QNX-supported target device or virtual machine	Running and testing the QNX application
6	Monitor / Display	Viewing the hospital management dashboard

Methodology and Module Description

1. Methodology

The **Hospital Emergency Response Coordinator** using QNX RTOS follows a process-based approach to manage emergency patients efficiently. The system uses five independent processes connected through Inter-Process Communication (IPC). Patient details are recorded, patients are classified into four severity levels—Critical, Severe, Moderate, and Mild—and ranked according to priority. The system monitors ICU bed availability and displays patient rankings and resource status on a dashboard. QNX RTOS supports process execution and coordination, helping improve patient management and timely decision-making.

2. Module Description

1. Patient Management Process: Records patient details and manages incoming patient information.

2. Triage Process: Classifies patients according to their severity level: Critical, Severe, Moderate, or Mild.

3. Priority Process: Ranks patients based on severity and identifies the next patient to be attended to.

4. Resource Management Process: Monitors ICU bed availability and maintains resource status information.

5. Display Process: Displays patient priority rankings, the next patient to be treated, and ICU bed availability on the dashboard.

Implementation

The **Hospital Emergency Response Coordinator** is implemented using QNX RTOS with five independent processes: Patient Management, Triage, Priority, Resource Management, and Display. Patient details are entered, severity levels are assigned, and patients are ranked according to priority. ICU bed availability is monitored, and the results are displayed on the dashboard. IPC is used for communication between processes.

Source Code (C)

(i) triage.h

```
#ifndef TRIAGE_H

#define TRIAGE_H

#include <stdint.h>

#include <sys/iofunc.h>

#include <sys/dispatch.h>

typedef enum{CTAS_1=1,CTAS_2=2,CTAS_3=3,CTAS_4=4,CTAS_5=5}ctas_level_t;

typedef struct{int patient_id;int age;int heart_rate;int systolic_bp;int oxygen_sat;int temperature;int pain_level;char symptoms[64];}patient_msg_t;

typedef struct{int available_beds;int available_doctors;int available_nurses;int ventilators_free;int or_rooms_free;}resource_msg_t;

typedef struct{uint16_t type;patient_msg_t patient;}triage_request_t;

typedef struct{uint16_t type;patient_msg_t patient;resource_msg_t resources;}priority_request_t;

typedef struct{ctas_level_t level;char department[32];char action[64];int estimated_wait_min;}priority_reply_t;

typedef struct{uint16_t type;int patient_id;ctas_level_t level;char department[32];char action[64];int estimated_wait_min;}display_msg_t;

#endif
```

(ii) patient_management.c

```
#include <stdio.h>

#include <string.h>

#include "triage.h"
```

```
int main(void){
triage_request_t request={0};
printf("Enter Patient ID: ");scanf("%d",&request.patient.patient_id);
printf("Enter Age: ");scanf("%d",&request.patient.age);
printf("Enter Heart Rate: ");scanf("%d",&request.patient.heart_rate);
printf("Enter Systolic BP: ");scanf("%d",&request.patient.systolic_bp);
printf("Enter Oxygen Saturation: ");scanf("%d",&request.patient.oxygen_sat);
printf("Enter Temperature (x10 Celsius): ");scanf("%d",&request.patient.temperature);
printf("Enter Pain Level (0-10): ");scanf("%d",&request.patient.pain_level);
printf("Enter Symptoms: ");scanf(" %63[^\n]",request.patient.symptoms);
printf("Patient Registered Successfully\nPatient ID: %d\nSymptoms:
%s\n",request.patient.patient_id,request.patient.symptoms);
return 0;
}
```

(iii) triage.c

```
#include <stdio.h>
#include "triage.h"
ctas_level_t assess_patient(patient_msg_t p){
if(p.oxygen_sat<90||p.heart_rate>140||p.systolic_bp<90)return CTAS_1;
if(p.oxygen_sat<94||p.heart_rate>120||p.pain_level>=8)return CTAS_2;
if(p.pain_level>=5)return CTAS_3;
if(p.pain_level>=2)return CTAS_4;
return CTAS_5;
}
int main(void){
patient_msg_t patient={0};
printf("Enter Patient ID: ");scanf("%d",&patient.patient_id);
printf("Enter Heart Rate: ");scanf("%d",&patient.heart_rate);
printf("Enter Systolic BP: ");scanf("%d",&patient.systolic_bp);
printf("Enter Oxygen Saturation: ");scanf("%d",&patient.oxygen_sat);
printf("Enter Pain Level: ");scanf("%d",&patient.pain_level);
printf("Demonstration Priority: CTAS-%d\n",assess_patient(patient));
return 0;
```

```
}
```

(iv) priority.c

```
#include <stdio.h>
#include <string.h>
#include "triage.h"
priority_reply_t calculate_priority(patient_msg_t patient,resource_msg_t resources){
priority_reply_t reply={0};
if(patient.oxygen_sat<90||patient.heart_rate>140||patient.systolic_bp<90){
reply.level=CTAS_1;strcpy(reply.department,"Emergency");strcpy(reply.action,"Immediate assessment");reply.estimated_wait_min=0;
}else if(patient.oxygen_sat<94||patient.heart_rate>120||patient.pain_level>=8){
reply.level=CTAS_2;strcpy(reply.department,"Emergency");strcpy(reply.action,"Urgent assessment");reply.estimated_wait_min=5;
}else{
reply.level=CTAS_3;strcpy(reply.department,"General");strcpy(reply.action,"Clinical assessment");reply.estimated_wait_min=30;
}
if(resources.available_doctors<=0)strcpy(reply.action,"Escalate: no available doctors");
return reply;
}
int main(void){
patient_msg_t patient={0};resource_msg_t resources={0};
printf("Enter Patient ID: ");scanf("%d",&patient.patient_id);
printf("Enter Heart Rate: ");scanf("%d",&patient.heart_rate);
printf("Enter Systolic BP: ");scanf("%d",&patient.systolic_bp);
printf("Enter Oxygen Saturation: ");scanf("%d",&patient.oxygen_sat);
printf("Enter Pain Level: ");scanf("%d",&patient.pain_level);
printf("Available Doctors: ");scanf("%d",&resources.available_doctors);
priority_reply_t reply=calculate_priority(patient,resources);
printf("Priority: CTAS-%d\nDepartment: %s\nAction: %s\nEstimated Wait: %d minutes\n",reply.level,reply.department,reply.action,reply.estimated_wait_min);
return 0;
}
```

(v) resource_management.c

```
#include <stdio.h>

#include "triage.h"

int main(void){

resource_msg_t resources={0};

printf("Enter Available ICU Beds: ");scanf("%d",&resources.available_beds);

printf("Enter Available Doctors: ");scanf("%d",&resources.available_doctors);

printf("Enter Available Nurses: ");scanf("%d",&resources.available_nurses);

printf("Enter Free Ventilators: ");scanf("%d",&resources.ventilators_free);

printf("Enter Available Operation Rooms: ");scanf("%d",&resources.or_rooms_free);

printf("\nHospital Resource Status\nICU Beds: %d\nDoctors: %d\nNurses: %d\nVentilators: %d\nOperation
Rooms:
%d\n",resources.available_beds,resources.available_doctors,resources.available_nurses,resources.ventilators_fre
e,resources.or_rooms_free);

return 0;

}
```

(vi) display.c

```
#include <stdio.h>

#include "triage.h"

int main(void){

display_msg_t message={0};int level;

printf("Enter Patient ID: ");scanf("%d",&message.patient_id);

printf("Enter CTAS Level (1-5): ");scanf("%d",&level);

if(level<1||level>5){printf("Invalid CTAS level.\n");return 1;}

message.level=(ctas_level_t)level;

printf("Enter Department: ");scanf("%31s",message.department);

printf("Enter Action: ");scanf(" %63[^\n]",message.action);

printf("Enter Estimated Wait (minutes): ");scanf("%d",&message.estimated_wait_min);

printf("\nHospital Dashboard\nPatient ID: %d\nPriority: CTAS-%d\nDepartment: %s\nAction: %s\nEstimated
Wait: %d
minutes\n",message.patient_id,message.level,message.department,message.action,message.estimated_wait_min)
;

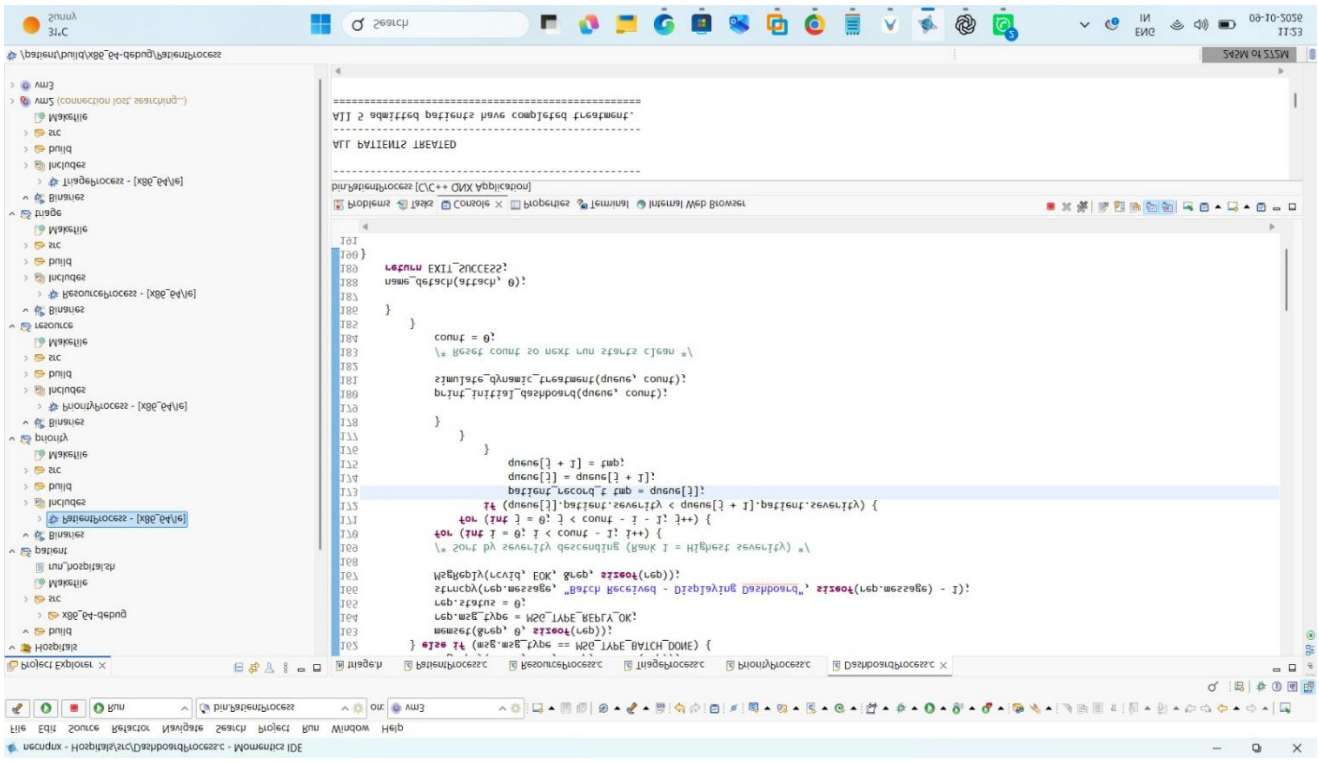
return 0;

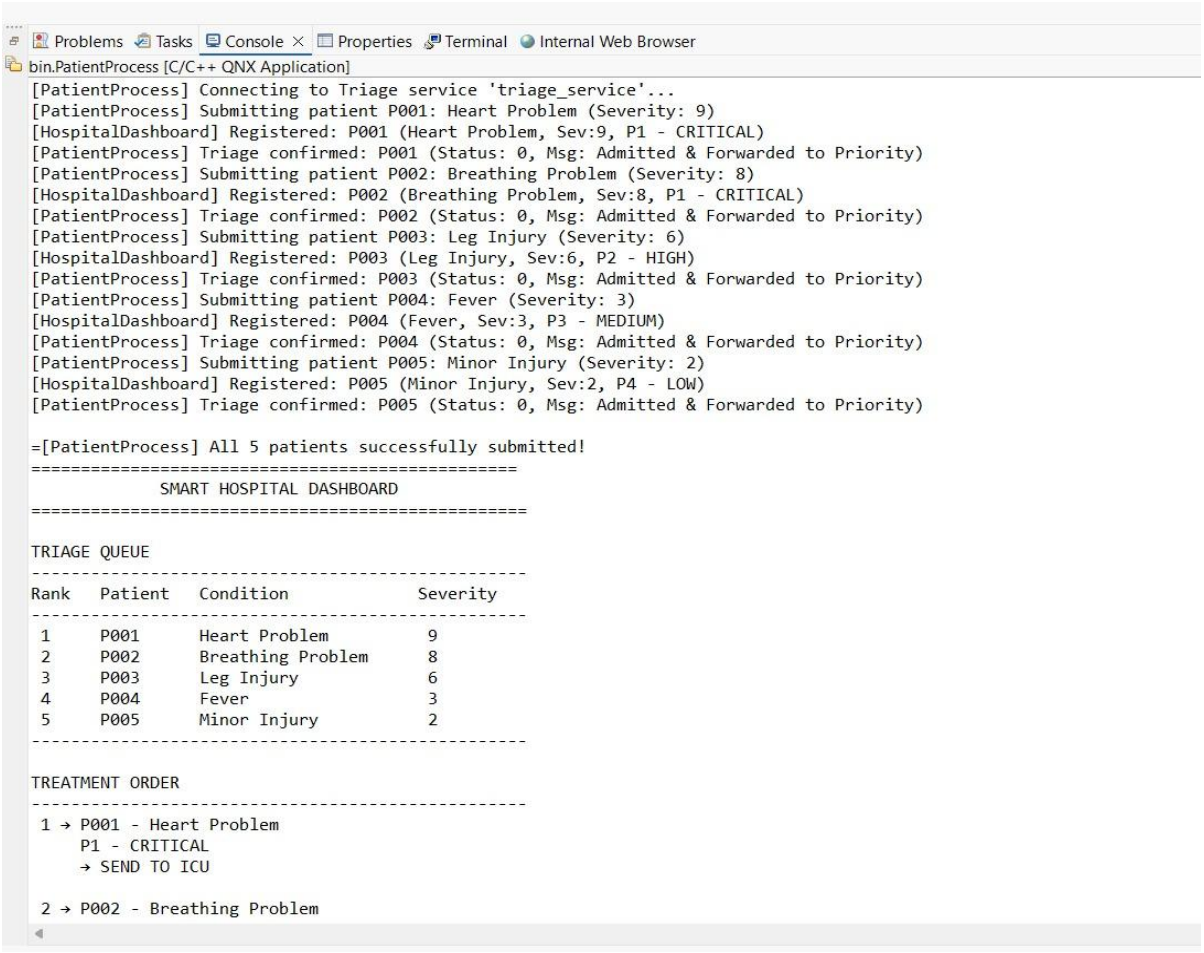
}
```

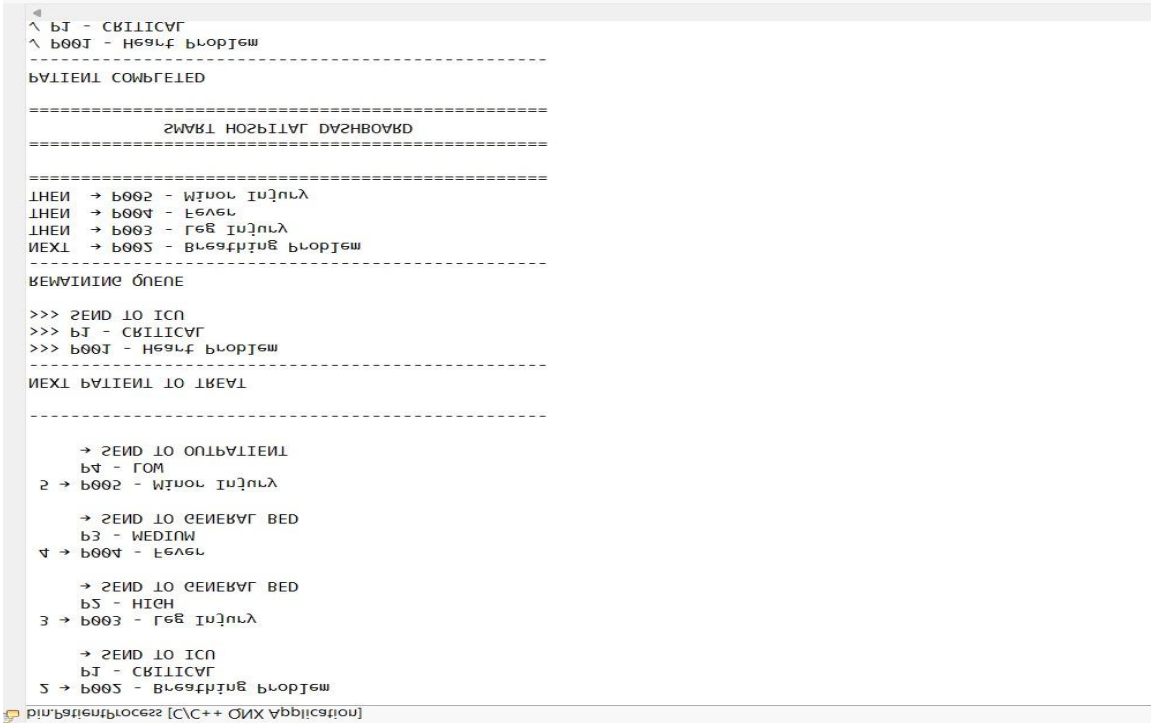
Result

The **Hospital Emergency Response Coordinator** displays the registered patient details, assigns a demonstration priority level, and shows hospital resource availability.

- Patient details are recorded successfully.
- Patients are classified into CTAS levels 1–5 using demonstration rules.
- Patient priority and estimated waiting time are displayed.
- ICU beds, doctors, nurses, ventilators, and operation rooms are monitored.
- The display module presents patient priority and department information.




Problems Tasks Console Properties Terminal Internal Web Browser
bin.PatientProcess [C/C++ QNX Application]
[PatientProcess] Connecting to Triage service 'triage_service'...
[PatientProcess] Submitting patient P001: Heart Problem (Severity: 9)
[HospitalDashboard] Registered: P001 (Heart Problem, Sev:9, P1 - CRITICAL)
[PatientProcess] Triage confirmed: P001 (Status: 0, Msg: Admitted & Forwarded to Priority)
[PatientProcess] Submitting patient P002: Breathing Problem (Severity: 8)
[HospitalDashboard] Registered: P002 (Breathing Problem, Sev:8, P1 - CRITICAL)
[PatientProcess] Triage confirmed: P002 (Status: 0, Msg: Admitted & Forwarded to Priority)
[PatientProcess] Submitting patient P003: Leg Injury (Severity: 6)
[HospitalDashboard] Registered: P003 (Leg Injury, Sev:6, P2 - HIGH)
[PatientProcess] Triage confirmed: P003 (Status: 0, Msg: Admitted & Forwarded to Priority)
[PatientProcess] Submitting patient P004: Fever (Severity: 3)
[HospitalDashboard] Registered: P004 (Fever, Sev:3, P3 - MEDIUM)
[PatientProcess] Triage confirmed: P004 (Status: 0, Msg: Admitted & Forwarded to Priority)
[PatientProcess] Submitting patient P005: Minor Injury (Severity: 2)
[HospitalDashboard] Registered: P005 (Minor Injury, Sev:2, P4 - LOW)
[PatientProcess] Triage confirmed: P005 (Status: 0, Msg: Admitted & Forwarded to Priority)
=[PatientProcess] All 5 patients successfully submitted!
SMART HOSPITAL DASHBOARD
TRIAGE QUEUE
Rank Patient Condition Severity
1 P001 Heart Problem 9
2 P002 Breathing Problem 8
3 P003 Leg Injury 6
4 P004 Fever 3
5 P005 Minor Injury 2
TREATMENT ORDER
1 → P001 - Heart Problem
P1 - CRITICAL
→ SEND TO ICU
2 → P002 - Breathing Problem


bin.PatientProcess [C/C++ QNX Application]
5 → P005 - Breathing Problem
P1 - CRITICAL
→ SEND TO ICU
3 → P003 - Leg Injury
P2 - HIGH
→ SEND TO GENERAL BED
4 → P004 - Fever
P3 - MEDIUM
→ SEND TO GENERAL BED
2 → P002 - Minor Injury
P4 - LOW
→ SEND TO OUTPATIENT
NEXT PATIENT TO TREAT
>>> P001 - Heart Problem
>>> P1 - CRITICAL
>>> SEND TO ICU
REMAINING QUEUE
NEXT → P005 - Breathing Problem
THEN → P003 - Leg Injury
THEN → P004 - Fever
THEN → P002 - Minor Injury
SMART HOSPITAL DASHBOARD
PATIENT COMPLETED
1 → P001 - Heart Problem
P1 - CRITICAL

```
==================================================
              SMART HOSPITAL DASHBOARD
==================================================

PATIENT COMPLETED
--------------------------------------------------
✓ P001 - Heart Problem
✓ P1 - CRITICAL
✓ ICU treatment started
--------------------------------------------------

NEXT PATIENT TO TREAT
--------------------------------------------------
>>> P002 - Breathing Problem
>>> P1 - CRITICAL
>>> SEND TO ICU

REMAINING QUEUE
--------------------------------------------------
1 → P003               Leg Injury    P2 - HIGH
2 → P004               Fever         P3 - MEDIUM
3 → P005               Minor Injury P4 - LOW
==================================================

==================================================
              SMART HOSPITAL DASHBOARD
==================================================

PATIENT COMPLETED
--------------------------------------------------
✓ P002 - Breathing Problem
✓ P1 - CRITICAL
✓ ICU treatment started
--------------------------------------------------

NEXT PATIENT TO TREAT
--------------------------------------------------
>>> P003 - Leg Injury
>>> P2 - HIGH
>>> SEND TO GENERAL BED
```

Conclusion

The **Hospital Emergency Response Coordinator** using QNX RTOS demonstrates a method for prioritizing emergency patients based on severity and monitoring hospital resources. The system includes patient management, triage, priority ranking, resource management, and dashboard display modules. It aims to improve patient monitoring, resource utilization, and timely decision-making.

Future Scope

- Multi-Department Support: Extend the system to manage multiple hospital departments.
- Real-Time IPC Integration: Connect all modules using QNX inter-process communication.
- Advanced Dashboard: Develop a graphical dashboard for live patient and resource updates.
- Resource Expansion: Support tracking additional ICU beds, ventilators, and medical equipment.
- Multi-Hospital Integration: Extend the system to share information between connected hospitals.
- Performance Optimization: Evaluate and improve response time and process scheduling.

“The system improves emergency patient prioritization and hospital resource management”.